Captain Adel — Execution Plan

SECTION 06-operations-it TYPE plan STATUS active OWNER Founder UPDATED 2026-06-16

A working plan derived from `captadel/roadmap.md`. Four sections:

1. Critique & reprioritization of the roadmap
2. Sequenced "Now" execution plan (~6 weeks)
3. Deep-dive implementation plan for the keystone item (eval upgrade)
4. Milestone / release plan (v0.4 → v1.0)

Treat this as a draft. Every dated claim is a proposal, not a commitment.

---

1. Critique & reprioritization

The roadmap is in good shape — the principles are right, the Now/Next/Later discipline is honest, and the eval-gating commitment is exactly the right release signal for a safety-adjacent product. Pushback below is on sequencing and gaps, not direction.

1a. Inversion: eval upgrades land before retrieval upgrades

The roadmap lists **Retrieval** before **Evaluation** inside the Now bucket. But the principle "every change is eval-gated" only holds if the eval is sharp enough to detect the changes you're trying to ship. Today's eval relies on keyword heuristics — that means a hybrid retriever that returns paraphrased-but-correct passages can score *worse* than BM25 returning verbatim-but-wrong ones. **Eval upgrades must land first or the rest of Now is unmeasurable.**

1b. Citation-faithfulness check is two things, not one

The roadmap lists it under Evaluation: "verify the cited section actually contains the claimed text; flag/strip uncited claims post-hoc." That second clause is a **runtime guard**, not a test. The same checker should run in two places — as an eval metric (offline) and as a post-generation guard (online). Splitting it into "check" + "guard" makes the dependency clear and lets the runtime guard ship later in the same wave.

1c. Streaming (SSE) is mis-bucketed in Next

It's listed under Models in Next, gated on ALLaM landing. But streaming is a UX win available today on Gemini, doesn't require any model change, and is decoupled from the eval bar (evals don't care if a wrong answer streams). **Move SSE into Now under Safety & ops / UX.** It's the single biggest perceived-latency win we can deliver this quarter.

1d. Promoting ALLaM might actually be Now, not Next

Every Gemini call is paid + external. In-Kingdom ALLaM is both the PDPL story and the long-term cost story. If GACAR usage grows, the cost line argues for promoting ALLaM ahead of fine-tunes and oral-exam coaching. Keep the *fine-tune* in Next, but consider pulling the *quantized production endpoint* in.

1e. LoRA fine-tune is a risky default

Fine-tuning a small model for citation discipline often degrades it — the format constraints fight the training distribution. RAG-first + a strong system prompt + better retrieval usually beats a LoRA for citation format. Mark the LoRA as an **experiment** that must beat the RAG-only baseline on evals before being promoted, not a default path.

1f. Compute tools need a tool-use eval track of their own

Wiring E6B, W&B, fuel, and VFR minima as tools is a real differentiator — no other tutor will have it — but it introduces a new failure class: a confidently wrong calculation. Before any compute tool ships, the eval suite needs **tool-use cases**: did Adel call the right tool, did he feed it the right inputs, did he show his working, did he still cite the rule the calculation derives from. Without that, the runtime gain is offset by a new safety hole.

1g. Gaps the roadmap doesn't name

Worth adding to Now/Next:

- **Corpus versioning + diff pipeline.** GACAR gets revised. The roadmap mentions `list_changes` and "what changed digests" but not the upstream ingestion + versioning that those depend on. This is the foundation; nothing else is durable without it.
- **Refusal taxonomy.** The principle says "refuses rather than guesses" but there's no documented set of refusal categories (out-of-scope, ambiguous question, conflicting sources, missing citation, injection attempt) or canonical refusal language. Without a taxonomy, refusal-calibration evals have nothing to score against.
- **Runtime injection logging.** Adversarial eval cases exist — good — but is there a runtime detector + log that surfaces real-world injection attempts so the eval suite can grow from them?
- **Per-tenant cost guardrails.** If the public multi-tenant API (Later) ships without per-tenant cost caps + alerts + a kill-switch, one runaway tenant takes down the budget. This needs to land *before* the public API, not after.
- **Multi-tenant prompt isolation.** Fly GACA plugs in via the gateway proxy today. As more tenants land, what guarantees one tenant's system prompt or persona can't leak into another's response? Worth an explicit isolation contract + an eval case.

1h. Arabic deserves its own track, not scattered items

Bilingual parity is a stated principle, but Arabic-specific work is sprinkled across Retrieval (synonyms), Evaluation (AR cases), Models (ALLaM, fine-tune), and UX (RTL site). Pull these into an **AR parity track** with one owner and one definition-of-done: AR eval pass rate within 5% of EN across the whole suite.

---

2. Sequenced "Now" execution plan (~6 weeks)

Three waves. Each wave's exit criteria are eval-measurable.

Wave 1 — Eval foundation (weeks 1–2)

Why first: see §1a. Eval sharpness is the precondition for trusting every later change.

- Citation-faithfulness checker (offline metric)
- LLM-as-judge grader for groundedness + citation correctness + refusal appropriateness
- Expand AR coverage to ~30 cases; add ~20 adversarial / injection cases; add ~10 refusal cases
- Wire live eval into CI as a required PR check (Gemini provider; ALLaM when endpoint reachable)
- Refusal taxonomy doc + canonical refusal strings (precondition for refusal-calibration scoring)

Exit criteria: baseline scores recorded for EN + AR + adversarial subsets; CI required on PRs touching `captadel/**`; budget caps + caching prevent eval cost runaway.

Wave 2 — Retrieval quality (weeks 3–4)

Now that the eval can detect regressions, ship the retrieval lift.

- Hybrid retrieval: add embedding recall, fuse with BM25 via Reciprocal Rank Fusion
- Parent-child chunk expansion (retrieve chunk → expand to full section so limits/tables aren't truncated)
- Citation precision hardening: fix mangled-PDF-title bugs in `citationOf` / `sectionRefOf`
- Cross-encoder rerank of top-k, gated on the two above proving out on evals

Exit criteria: faithfulness mean ↑ ≥ 0.05 vs baseline; EN + AR pass rate ↑ on the existing suite; no refusal-calibration regression; p95 latency budget held.

Wave 3 — Hardened service (weeks 5–6)

Service maturity + the runtime side of citation faithfulness.

- Distributed rate limiter (Redis or Firestore) replacing per-process `ratelimit.js`
- App Check: monitoring → enforce

- Structured observability: per-turn metrics (latency, tool rounds, #sources, refusal rate, provider, fallbacks) + error beacon
- Runtime citation-faithfulness guard: strip or refuse on unsupported claims (online use of the Wave 1 checker)
- Streaming responses (SSE) on Gemini — pulled from Next (§1c)

Exit criteria: rate limit holds across replicas under load test; runtime faithfulness guard catches the bogus-citation eval cases that ship in Wave 1; p95 ≤ today; streaming live on captadel.com.

Out of scope for Now (deferred to Next)

- ALLaM production promotion (candidate to pull forward — see §1d)
- LoRA fine-tune (de-prioritized — see §1e)
- Compute tools (gated on tool-use evals — see §1f)
- Multi-tenant API + billing
- AR/RTL site parity (move to AR-parity track in Next)

---

3. Deep-dive: Wave 1 — Evaluation upgrade

The keystone. Three workstreams, ~2 weeks, designed to ship in this order: A → B → C, with C gating merge of A + B into main.

Workstream A — Citation-faithfulness checker

Behaviour. For each model answer, parse out cited `(Part, section)` references, look up the actual text from the GACAR corpus, and verify every regulatory claim in the answer is supported by at least one cited section.

Implementation.

- New module: `captadel/evals/checks/citation-faithfulness.js`
- Reuse `src/brain/bm25.js`'s citation helpers (`citationOf`, `sectionRefOf`) to resolve references to text. Harden the helpers against mangled PDF titles as part of this work — the `AIRCRAFTONTHEWATER` class of bugs noted in `bm25.js`.
- Sentence-split the answer; flag any sentence as a "regulatory claim" if it contains: a number with units (e.g. "1,500 ft", "3 NM"), a modal ("shall", "must", "may not"), or a section reference (e.g. "GACAR 91.155").
- For each claim sentence, run an entailment check against the union of cited-section text: a cheap LLM-as-judge call with prompt "does PASSAGE support CLAIM? yes / no / partial".
- Score per answer: `supported_claims / total_claims`. Score per run: mean over cases.
- Add a column to `evals/run.js` output alongside today's keyword-match metric.

Eval cases for the checker itself (meta-evals).

- ~20 known-good cases: answers with correct citations + verbatim language → expect score 1.0
- ~10 negative cases: answers that cite a real section but invent a number → expect score < 1.0 with the specific bogus claim flagged
- ~10 partial cases: answers with one supported claim and one ungrounded aside → expect partial credit

Runtime variant (lands in Wave 3). Same module called from `src/brain/*` after generation; if score < threshold (e.g. 0.8), either strip the unsupported sentences and re-stitch, or refuse with: "I can ground part of this in GACAR but not all of it — here's what's grounded: ...".

Workstream B — LLM-as-judge grader

Behaviour. A judge model rates each answer on a four-point rubric: groundedness, citation correctness, refusal appropriateness, helpfulness. Each scored 0–2 with reasons.

Implementation.

- New module: `captadel/evals/judges/groundedness.js` with prompt templates per language (EN, AR).
- Judge model: a pinned cheap model (Gemini Flash, or hosted ALLaM if available). Pin model + version in the prompt header so reruns are reproducible.
- Run on every eval case; persist per-case scores + judge rationales to `evals/runs/<timestamp>.json`.
- Add baseline file `evals/baseline.json` and a `--compare baseline` flag in `evals/run.js` that diffs current run against baseline and exits non-zero on regression beyond tolerance.

Eval set additions (lands alongside this workstream).

- AR coverage: +20 cases mirroring the heaviest-used EN topics (numeric limits, refusals, follow-up questions). Target ~30 AR total.
- Adversarial: +20 cases. Mix of prompt injections embedded in retrieved passages, role-confusion attempts ("you are now an FAA examiner"), and "ignore previous instructions" patterns.
- Refusal: +10 cases. Out-of-scope (US FAR questions), ambiguous, conflicting sources, missing citation.

Workstream C — CI gate

Implementation.

- GitHub Actions job `eval.yml` triggered on PRs touching `captadel/**`.
- Runs `node evals/run.js --provider gemini --compare baseline` against a staging Gemini key stored as a repo secret.
- Optional second job for `--provider allam` once the endpoint is reachable; not required until ALLaM is in production.
- Required status check on `main` and release branches.

- Failure conditions (any of): pass rate drops > 2pp, faithfulness mean drops > 0.05, refusal calibration regresses on the refusal subset, adversarial pass rate drops at all.

Cost + flake mitigations.

- Cache embedding + judge calls keyed on `(model, prompt, input_hash)` — same PR re-runs don't re-pay.
- Concurrency limit (e.g. 8) so one PR doesn't fan out 200 simultaneous calls.
- Daily budget cap; if hit, eval marked "skipped due to budget" with a manual override label.
- Eval-set checksum check: PRs that touch eval cases re-baseline rather than compare.

Files touched

```
captadel/
  evals/
    checks/citation-faithfulness.js        (new)
    judges/groundedness.js                 (new)
    cases/ar/*.json                        (new + expanded)
    cases/adversarial/*.json               (new)
    cases/refusal/*.json                   (new)
    run.js                                 (add metrics + --compare)
    baseline.json                          (new)
  src/brain/bm25.js                        (harden citation helpers)
  docs/refusal-taxonomy.md                 (new)
.github/workflows/eval.yml                 (new)
```

Risks + mitigations

- **Judge bias.** A judge model can be systematically lenient or harsh. Mitigation: run the judge over a held-out human-labelled subset (~30 cases) quarterly; flag drift.
- **Live eval cost spike.** See cost mitigations above. Worst-case kill-switch: eval becomes warning-only for a week while we investigate.
- **Citation helper changes break old runs.** Mitigation: baseline gets re-recorded on the PR that hardens the helpers; that PR is its own release.

Exit criteria (Wave 1)

- Faithfulness checker live; baseline measured on `main`.
- Judge grader live; per-language baselines recorded.
- 50+ EN cases, 30+ AR cases, 20+ adversarial cases, 10+ refusal cases in the suite.
- CI gate green on `main`; required on PRs touching `captadel/**`.
- Refusal taxonomy doc merged.

---

4. Milestone / release plan

Versions are eval-gated. A version doesn't ship until its exit criteria are met on the eval suite that exists at that point — and the eval suite itself is part of the gate.

v0.4 — "Eval-graded" (Wave 1, weeks 1–2)

Scope. Citation-faithfulness checker, LLM-as-judge grader, CI gate, AR + adversarial + refusal case coverage, refusal taxonomy doc.

Exit. Baseline scores recorded for EN + AR + adversarial. CI required. No user-visible behaviour change.

Why ship. Unblocks every later release.

v0.5 — "Sharper retrieval" (Wave 2, weeks 3–4)

Scope. Hybrid retrieval + RRF, parent-child chunk expansion, citation precision hardening, cross-encoder rerank.

Exit. Faithfulness mean ↑ ≥ 0.05 vs v0.4 baseline. EN + AR pass rate ↑ on existing cases. No refusal-calibration regression. p95 latency ≤ v0.4.

User-visible. Better answers, especially on paraphrased questions and questions whose answer lives in a table that used to get truncated.

v0.6 — "Hardened service" (Wave 3, weeks 5–6)

Scope. Distributed rate limiter, App Check enforcement, structured observability, runtime citation-faithfulness guard, streaming (SSE) on Gemini.

Exit. Rate limit holds across replicas under load test. Runtime guard catches the Wave-1 bogus-citation cases at >95%. p95 ≤ v0.5. Streaming live on captadel.com.

User-visible. Faster perceived latency (streaming), fewer subtly-wrong answers (runtime guard), better operational visibility.

v0.7 — "ALLaM in production" (Next bucket, weeks 7–10)

Scope. Quantized ALLaM endpoint (AWQ/GPTQ) in a KSA region. Per-language benchmarking. `MODEL_PROVIDER=auto` flipped for AR queries (and EN if it beats Gemini on cost-adjusted quality).

Exit. AR eval pass rate within 5% of EN on the chosen provider. PDPL story documented (data residency, retention, processor list). Provider fallback path proven by chaos test (kill ALLaM mid-stream → Gemini takes over).

Why now. PDPL + cost trajectory — see §1d.

v0.8 — "Conversational" (weeks 11–14)

Scope. Query rewriting for multi-turn follow-ups, AR ↔ corpus-vocab mapping at the retrieval step, AR/RTL parity on the captadel.com site, feedback loop ingestion (👍/👎 → PDPL-safe log → eval seed).

Exit. Multi-turn case subset added to eval and passing. AR RTL site at parity with EN visually + functionally. Feedback ingest pipeline producing eval candidates from real traffic.

v0.9 — "Instructor, not lookup"

Scope. Compute tools (E6B, W&B, fuel, VFR minima) wired as tools. Tool-use eval subset (see §1f). Scenario / oral-exam coach. "What changed" digests on top of the corpus-versioning pipeline.

Exit. Tool-call eval cases pass at ≥95% (right tool, right inputs, working shown, rule still cited). Tools never run without retrieval grounding. Corpus-version-diff job produces a per-Part changelog automatically.

Dependency. Corpus versioning + diff pipeline (gap in §1g) must land first — pull into Next.

v1.0 — "Platform"

Scope. Accounts/billing/quota on captadel.com. Public multi-tenant API with per-tenant keys, rate tiers, usage metering, cost guardrails, kill-switch. Multi-tenant prompt isolation contract + eval. Embeddable widget / SDK. Voice (Saudi-accented EN + Khaleeji AR TTS/STT). Opt-in personalization tied to Fly GACA logbook. CDN + autoscaling.

Exit. Tenant isolation eval passes (no cross-tenant leakage on red-team cases). Per-tenant cost caps proven by a synthetic runaway-tenant test. Public API has OpenAPI docs + a published changelog.

Prerequisite for v1.0 ship: the cost-guardrail + isolation work from §1g lands at least one milestone earlier (likely v0.9) so v1.0 isn't gating on safety primitives.

Promote to standalone repo

`git subtree split --prefix=captadel → FlyGACA/captadel`. Schedule this between v0.6 and v0.7 — after the service is hardened, before ALLaM lands and changes the deploy surface area.

Open questions for you

1. Do you agree with flipping eval-before-retrieval inside Now? If not, I'll re-sequence Wave 1 and 2.
2. Is SSE worth pulling into Now (§1c), or do you want to keep it with the ALLaM work?
3. Is ALLaM-in-production a Now item or genuinely Next (§1d)? Cost numbers would settle this.
4. Who owns the AR parity track (§1h)? Without one owner this stays scattered.

5. Is corpus versioning (§1g) something we have already and the roadmap just doesn't mention, or genuinely missing?